

Protocolo HTTP

Mensajes, métodos, códigos de estado y cabeceras

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 SRI · Servicios de Red e Internet

SRI · UNIDAD 2 — PROTOCOLO HTTP

01

El protocolo HTTP

Mensajes, cabeceras y estado

¿Qué es HTTP?

" ***HyperText Transfer Protocol** es el protocolo de la **capa de aplicación** que permite la comunicación entre clientes, servidores y proxies en la web.*

CARACTERÍSTICAS FUNDAMENTALES

- Basado en el esquema **petición / respuesta**: el cliente pide, el servidor responde
- Versiones en uso: **HTTP/1.1**, **HTTP/2** y **HTTP/3**; cambian la forma de transportar los mensajes, no su significado
- En **HTTP/1.1** los mensajes son **texto legible**; HTTP/2 y HTTP/3 son **binarios**
- Protocolo **sin estado**: el servidor no recuerda quién hizo cada petición

 La falta de estado se compensa más tarde con **cookies** y **sesiones**.

El modelo TCP/IP: cuatro niveles

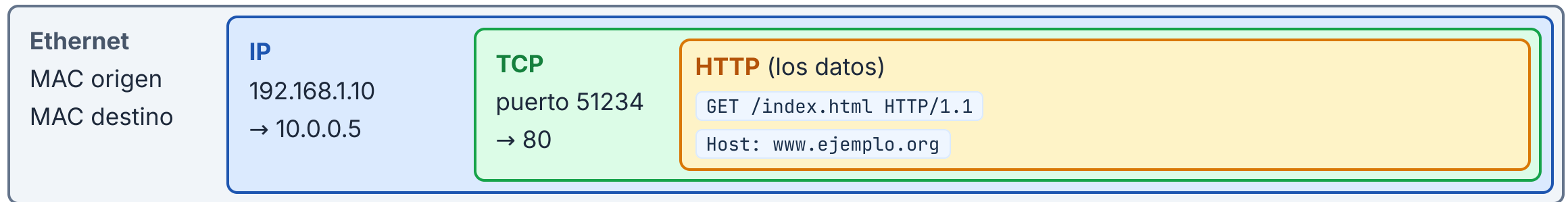
Cada nivel resuelve **un problema** y tiene **su propia dirección**. Cada uno se apoya en el de abajo.

Nivel	¿Qué resuelve?	Dirección	Protocolos	¿Quién lo hace?
Aplicación	¿Qué se pide y qué se responde?	URL, cabecera <code>Host</code>	HTTP, DNS, SMTP	El programa: <code>curl</code> , Apache
Transporte	¿A qué programa del equipo va?	Puerto	TCP, UDP	Sistema operativo
Red	¿A qué equipo va y por qué camino?	IP	IP	Sistema operativo y routers
Enlace	¿A qué tarjeta de la red local va?	MAC	Ethernet, Wi-Fi	Tarjeta de red y switch

i HTTP solo se ocupa de **qué se pide**. Que el mensaje llegue al equipo y al programa correctos es trabajo de los niveles de abajo.

Encapsulamiento

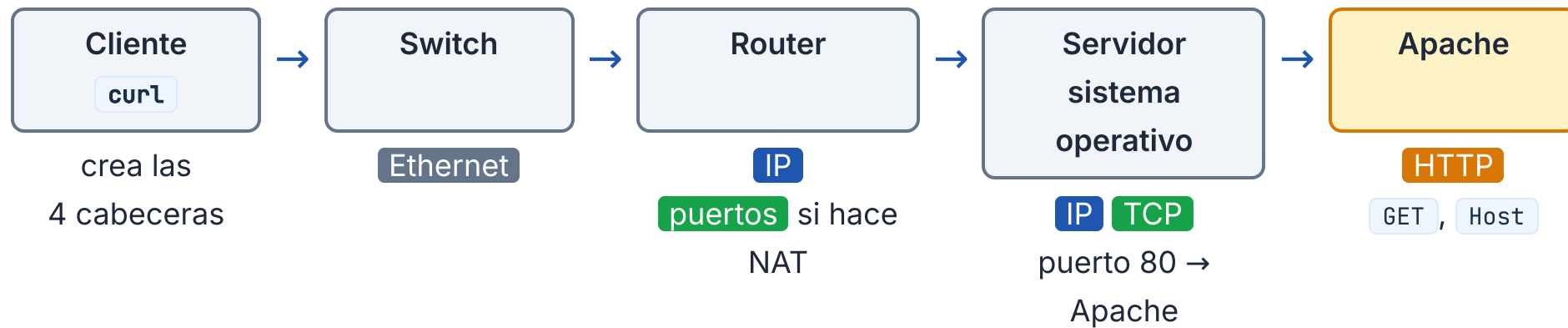
Al enviar, cada nivel **añade su cabecera** a lo que le entrega el nivel de arriba. Así viaja una petición HTTP:



- Al **recibir** se hace al revés: cada nivel **quita su cabecera** y pasa el resto al nivel de arriba
- Cada nivel **solo lee su cabecera**: lo que va dentro son datos que no entiende
- El nivel de red ve IPs; el de transporte, puertos. **Solo la aplicación ve el** GET **y el** Host

¿Quién mira cada cabecera?

El recorrido de la petición anterior hasta el servidor web:



- El **router nunca ve la URL ni el `Host`**: decide solo con IPs (y puertos si hace NAT)
- El sistema operativo del servidor entrega los datos al programa que **escucha en el puerto 80**: `ss -tlnp` lo muestra
- Solo Apache **entiende la petición**: por eso un **virtual host** puede elegir el sitio por el nombre

HTTP sobre TCP

HTTP es un protocolo de **aplicación**: se apoya en **TCP** (transporte) para que sus mensajes lleguen.

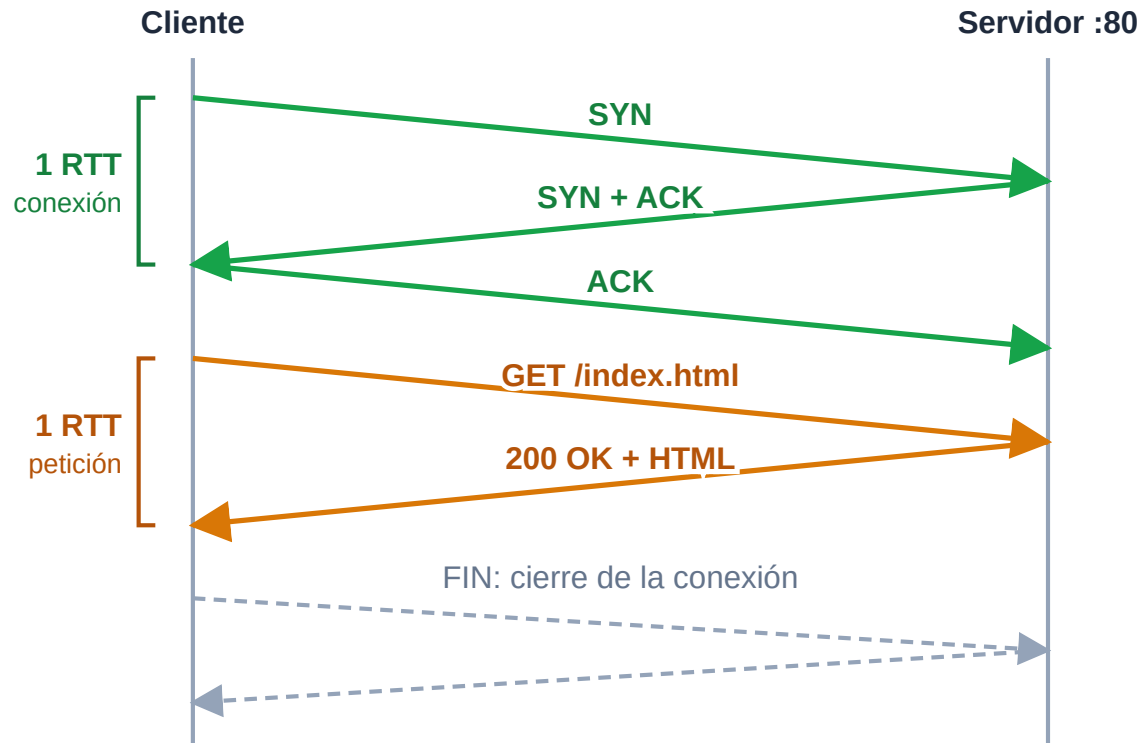
LO QUE TCP LE DA A HTTP

- Entrega **fiable y en orden**: si se pierde un segmento, TCP lo reenvía
- **Orientado a conexión**: antes de enviar la petición hay que **establecer la conexión**
- Puertos: **80** (HTTP) y **443** (HTTPS)

HTTP/3 CAMBIA EL TRANSPORTE

- Usa **QUIC** sobre **UDP 443**
- QUIC hace el trabajo de TCP (fiabilidad, orden) y **establece la conexión más rápido**
- Los mensajes HTTP **significan lo mismo**

Establecimiento de la conexión TCP



- **Transporte (TCP): saludo en tres pasos** (*three-way handshake*). Ningún dato HTTP viaja hasta que termina
- **Aplicación (HTTP):** la petición y la respuesta van **dentro** de esa conexión
- **RTT** (*round-trip time*): lo que tarda un mensaje en ir y volver. Lo ves con `ping`
- Cada conexión nueva cuesta **1 RTT más** antes de poder pedir nada

Conexiones persistentes (*keep-alive*)

Una página con **3 recursos** (`index.html` , `estilo.css` , `logo.png`), con un RTT de 50 ms:

Sin keep-alive (HTTP/1.0): una conexión por recurso → **6 RTT ≈ 300 ms**



Con keep-alive (HTTP/1.1): una conexión para todas las peticiones → **4 RTT ≈ 200 ms**



tiempo →

- Se ahorra **un saludo TCP por recurso**: con 50 recursos, 100 RTT frente a 51
- El servidor también gana: abre y cierra **menos conexiones**
- En HTTP/1.1 es lo normal: lo indica la cabecera `Connection: keep-alive` , y el servidor cierra la conexión si no recibe peticiones durante un tiempo

 Con `curl -v <url1> <url2>` (mismo servidor) verás que la segunda petición **reutiliza la conexión**.

Versiones de HTTP

VERSIÓN	AÑO	TRANSPORTE	FORMATO	TRÁFICO WEB (APROX.)
HTTP/1.1	1997	TCP	Texto	~30 %
HTTP/2	2015	TCP + TLS	Binario, varias peticiones a la vez por una conexión, cabeceras comprimidas	~50 %
HTTP/3	2022	QUIC (UDP) con TLS 1.3	Binario	~20 %

- La **semántica es la misma** en las tres: métodos, códigos de estado y cabeceras
- HTTP/1.1 sigue muy presente: servidores pequeños y comunicación **proxy → backend**
- Con `curl -I` a un sitio moderno verás `HTTP/2 200` (sin descripción y cabeceras en minúscula)

 Porcentajes aproximados del tráfico web en 2026 (Cloudflare Radar).

Anatomía de una URL

```
https://www.ejemplo.com:8080/docs/manual.html?idioma=es&pagina=2#capitulo2
```

- **Esquema** `https` — protocolo (y puerto por defecto: 80 / 443)
- **Host** `www.ejemplo.com` — servidor; viaja en la cabecera **Host**
- **Puerto** `8080` — opcional
- **Ruta** `/docs/manual.html` y **consulta** `?idioma=es&pagina=2` — viajan en la **línea de petición**
 - `?` inicia la consulta, `&` separa los parámetros y `=` separa nombre y valor
- **Fragmento** `#capitulo2` — **no se envía**: lo usa el navegador

```
GET /docs/manual.html?idioma=es&pagina=2 HTTP/1.1  
Host: www.ejemplo.com:8080
```

Estructura de los mensajes

PETICIÓN

- **Línea inicial:** método, URL del recurso y versión
- **Cabeceras** de petición (clave-valor)
- **Línea vacía**
- **Cuerpo** opcional (típico en `POST` y `PUT`)

```
GET /index.html HTTP/1.1  
Host: www.ejemplo.com  
User-Agent: curl/8.14.1
```

RESPUESTA

- **Línea de estado:** versión, código y descripción
- **Cabeceras** de respuesta
- **Línea vacía**
- **Cuerpo** con el recurso solicitado

```
HTTP/1.1 200 OK  
Content-Type: text/html  
Content-Length: 1024
```

Métodos de petición

- **GET** — Solicita un recurso. Los datos viajan en la URL (*query string*)
- **HEAD** — Igual que `GET` pero pide **sólo las cabeceras**
- **POST** — Envía datos al servidor en el **cuerpo** del mensaje
- **PUT** — Almacena el documento enviado en la URL indicada
- **DELETE** — Elimina el recurso referenciado por la URL
- **OPTIONS, PATCH, TRACE, CONNECT** — usos más específicos



`GET` y `HEAD` son métodos **seguros**: no deben modificar nada en el servidor. Es la **aplicación** la que tiene que respetarlo.

GET: los datos viajan en la URL

```
GET /login.php?usuario=pepe&clave=1234 HTTP/1.1  
Host: www.ejemplo.com
```

Los datos forman parte de la URL, así que quedan en el **historial** del navegador y en el `access.log` del servidor:

```
192.168.1.20 - - [06/Oct/2026:10:15:32 +0200] "GET /login.php?usuario=pepe&clave=1234 HTTP/1.1" 200 512
```

 Nunca se envían contraseñas ni datos personales por `GET`: cualquiera que lea los logs los ve.

POST: los datos viajan en el cuerpo

```
POST /login.php HTTP/1.1
Host: www.ejemplo.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 23

usuario=pepe&clave=1234
```

Los datos van **después de la línea vacía**. En el `access.log` solo aparece la ruta:

```
192.168.1.20 - - [06/Oct/2026:10:16:05 +0200] "POST /login.php HTTP/1.1" 200 512
```



POST no cifra nada: con `curl -v` o capturando el tráfico se ve el cuerpo. Para protegerlo hace falta **HTTPS**.

Códigos de estado

Cifra de tres dígitos que clasifica la respuesta. La primera cifra indica el grupo:

CÓDIGO	FAMILIA	SIGNIFICADO
1xx	Informativo	Información intermedia
2xx	Éxito	La petición se procesó correctamente (200 OK , 201 Created)
3xx	Redirección	El recurso está en otra URL (301 , 302 , 304)
4xx	Error del cliente	Petición incorrecta (400 , 401 , 403 , 404)
5xx	Error del servidor	Fallo en el servidor (500 , 502 , 503)

Códigos que verás administrando un servidor

3XX Y 4XX

- **301 / 302** — redirección
- **304** — se usa la copia en caché
- **401** — faltan credenciales
- **403** — acceso prohibido
- **404** — el recurso no existe

5XX

- **500** — fallo de la aplicación o la configuración
- **502** — el proxy no recibe respuesta válida del backend
- **503** — sin servicio (backends caídos)
- **504** — el backend tarda demasiado

Cabeceras HTTP

Información de la forma **clave: valor** asociada a peticiones y respuestas.

GENÉRICAS Y DE PETICIÓN

- `Host` — nombre del sitio solicitado
- `User-Agent` — cliente que hace la petición
- `Accept` — tipos MIME aceptados
- `Accept-Language` — idiomas
- `Cookie` — cookies enviadas
- `Authorization` — credenciales

DE RESPUESTA

- `Content-Type` — tipo MIME del cuerpo
- `Content-Length` — tamaño en bytes
- `Server` — software (y versión) del servidor
- `Set-Cookie` — define una cookie
- `Location` — URL destino en redirecciones
- `Cache-Control` — directivas de caché

 `Host` es **obligatoria** en HTTP/1.1: gracias a ella un servidor con una sola IP aloja varios sitios (**virtual hosts**). `Server` revela el software y la versión: en producción se suele ocultar.

Una petición real con `curl -v`

```
* Connected to dit.gonzalonazareno.org (5.196.224.198) port 80
> GET / HTTP/1.1
> Host: dit.gonzalonazareno.org
> User-Agent: curl/8.14.1
> Accept: */*
>
< HTTP/1.1 301 Moved Permanently
< Server: nginx/1.22.1
< Content-Type: text/html
< Content-Length: 169
< Connection: keep-alive
< Location: https://dit.gonzalonazareno.org/
```

- `*` información de `curl`: la conexión TCP (**red y transporte**)
- `>` envía el cliente · `<` responde el servidor: el mensaje HTTP (**aplicación**)
- La **línea vacía** cierra las cabeceras · **301** con `Location`: el recurso está en HTTPS

Cookies

" Pequeños fragmentos de información que el **navegador** guarda a petición del **servidor** mediante la cabecera `Set-Cookie` .

¿PARA QUÉ SIRVEN?

- Guardar información de **sesión**
- **Comercio electrónico**: carrito de la compra
- **Personalización**: idioma, tema, preferencias
- **Seguimiento** de visitas y publicidad
- Mantener la **sesión iniciada** (login)



Las cookies permiten **mantener estado** entre peticiones que, por diseño, son independientes.

Cookies: cómo funcionan

1ª RESPUESTA DEL SERVIDOR

```
< Set-Cookie: sesion=a3f9c2; Path=/;  
Max-Age=3600; HttpOnly; Secure
```

PETICIONES SIGUIENTES DEL NAVEGADOR

```
> Cookie: sesion=a3f9c2
```

ATRIBUTOS

- **Max-Age** / **Expires** — cuándo caduca
- **Path** / **Domain** — a qué URLs se envía
- **HttpOnly** — JavaScript no puede leerla
- **Secure** — solo se envía por HTTPS

Sesiones

HTTP es **stateless**: las sesiones añaden la noción de estado a las aplicaciones web.

LO QUE GUARDA EL SERVIDOR

- **Identificador** de sesión
- **Usuario** asociado
- **Tiempo de expiración**
- Datos temporales de la aplicación

LO QUE GUARDA EL CLIENTE

- Sólo el **identificador** de sesión
- Habitualmente en una **cookie**
- En cada petición lo envía al servidor
- El servidor recupera el resto de su lado

02

Aplicaciones que usan HTTP

Servidor web, proxy, proxy inverso y balanceador

Servidor web

Programa que **implementa el protocolo HTTP**: recibe las peticiones de los clientes y les devuelve los recursos solicitados.

- Sirve **páginas estáticas** (HTML, CSS, JS, imágenes...)
- Sirve **páginas dinámicas** generadas por lenguajes como **PHP, Python, Java...**, normalmente apoyándose en un **servidor de aplicaciones**
- Implementa funcionalidades del protocolo: **virtual hosts, redirecciones, autenticación, control de acceso...**



Ejemplos: **Apache** y **Nginx**, que estudiaremos en la siguiente presentación.

Proxy

Servidor que **intermedia las peticiones** entre los clientes de una red y los servidores de Internet. Actúa **en nombre de los clientes**.



- Da salida a equipos que **no acceden directamente a Internet**
- **Filtra** el tráfico HTTP: por dominio, URL, contenido, horario...
- Hace de **caché** de respuestas para acelerar accesos posteriores

 Software clásico: **Squid**.

Proxy inverso

Recibe las peticiones de los clientes y las **reenvía a uno o varios servidores internos** (backends). Actúa **en nombre de los servidores**: el cliente no accede directamente a ellos.



- **Protege** los servidores internos (ocultando su ubicación)
- Publica **varias aplicaciones** con un único nombre o IP
- Centraliza **TLS/SSL**, caché, compresión y autenticación

 Ejemplos: **Apache**, **Nginx**, **Varnish**, **Traefik**.

Nivel de red o nivel de aplicación

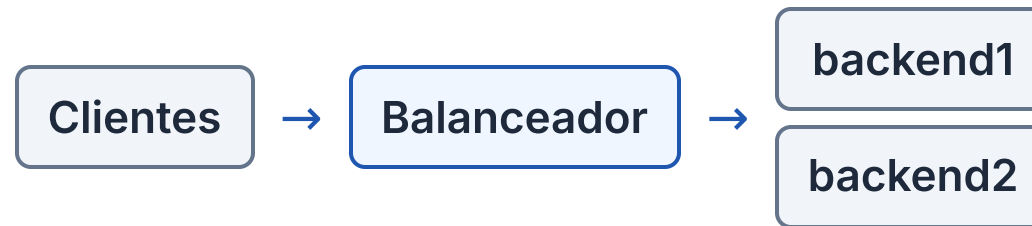
En la Unidad 1 usamos **DNAT** para llegar a un servidor interno, y el **proxy inverso** también lo hace. Parecen lo mismo, pero no trabajan en el mismo nivel:

	DNAT (IPTABLES / NFTABLES EN EL ROUTER)	PROXY INVERSO (APACHE, NGINX)
Nivel	Red y transporte	Aplicación
Decide por	IP y puerto de destino	Cabecera <code>Host</code> , URL
Conexiones TCP	Una : del cliente al servidor	Dos : del cliente al proxy y del proxy al servidor
¿Entiende HTTP?	No: cambia direcciones y reenvía	Sí: lee y cambia cabeceras, puede guardar en caché
Varios sitios en el puerto 80	No puede distinguirlos	Los reparte por nombre

 Pregúntate **qué mira para decidir**: si va **dentro de la petición HTTP** (`Host` , URL), es el nivel de **aplicación**.

Balanceador de carga

Distribuye las peticiones entre **varios backends que sirven lo mismo**. Normalmente es un **proxy inverso con varios backends**.



- Reparte la carga según un **algoritmo de distribución**
- Evita la **sobrecarga** de un único servidor
- Si cae un backend, sigue dando servicio con el resto

i Ejemplos: **Nginx**, **HAProxy**, **Apache mod_proxy_balancer**.

SRI · UNIDAD 2 — PROTOCOLO HTTP

¡Gracias!

HTTP

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 SRI